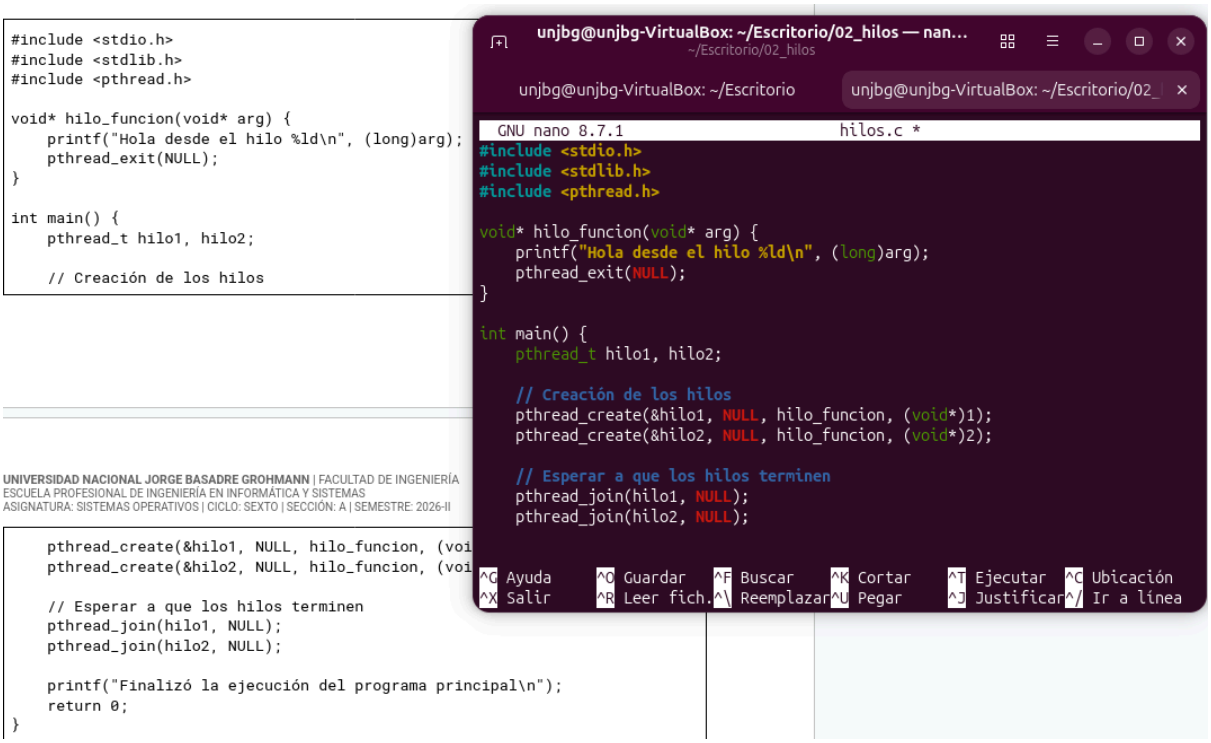
Práctica: “GESTIÓN DE HILOS”

Elaborado por: Giorgini Disalvatore Pacci Pérez 2024-119035

Desarrollo

Parte 1: Creación y gestión de hilos en C



```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* hilo_funcion(void* arg) {
    printf("Hola desde el hilo %ld\n", (long)arg);
    pthread_exit(NULL);
}

int main() {
    pthread_t hilo1, hilo2;

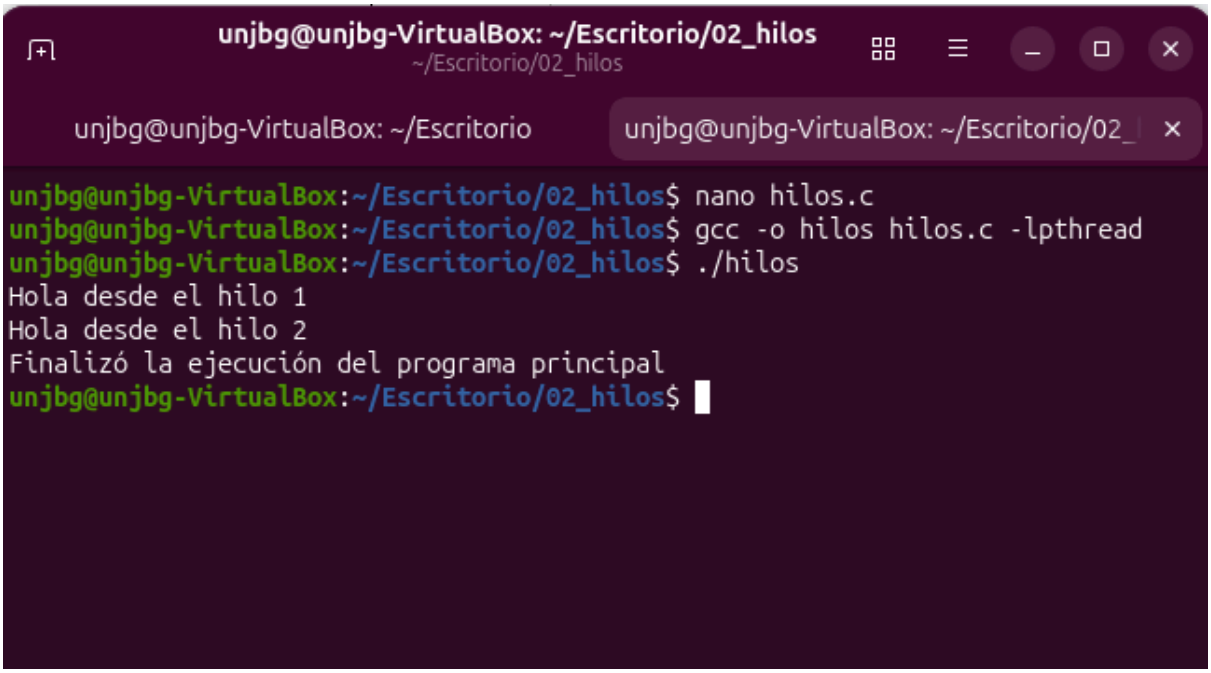
    // Creación de los hilos
    pthread_create(&hilo1, NULL, hilo_funcion, (void*)1);
    pthread_create(&hilo2, NULL, hilo_funcion, (void*)2);

    // Esperar a que los hilos terminen
    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);

    printf("Finalizó la ejecución del programa principal\n");
    return 0;
}
```


UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN | FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS
ASIGNATURA: SISTEMAS OPERATIVOS | CICLO: SEXTO | SECCIÓN: A | SEMESTRE: 2026-II

Nota. Abrimos una terminal y con ayuda del editor de texto nano lo guardamos



Nota. Compilación y ejecución del programa

```
unjbg@unjbg-VirtualBox:~/Escritorio/02_hilos$ time ./hilos
Hola desde el hilo 2
Hola desde el hilo 1
Finalizó la ejecución del programa principal

real    0m0,002s
user    0m0,000s
sys     0m0,003s
unjbg@unjbg-VirtualBox:~/Escritorio/02_hilos$
```

Nota. Registro del tiempo de ejecución

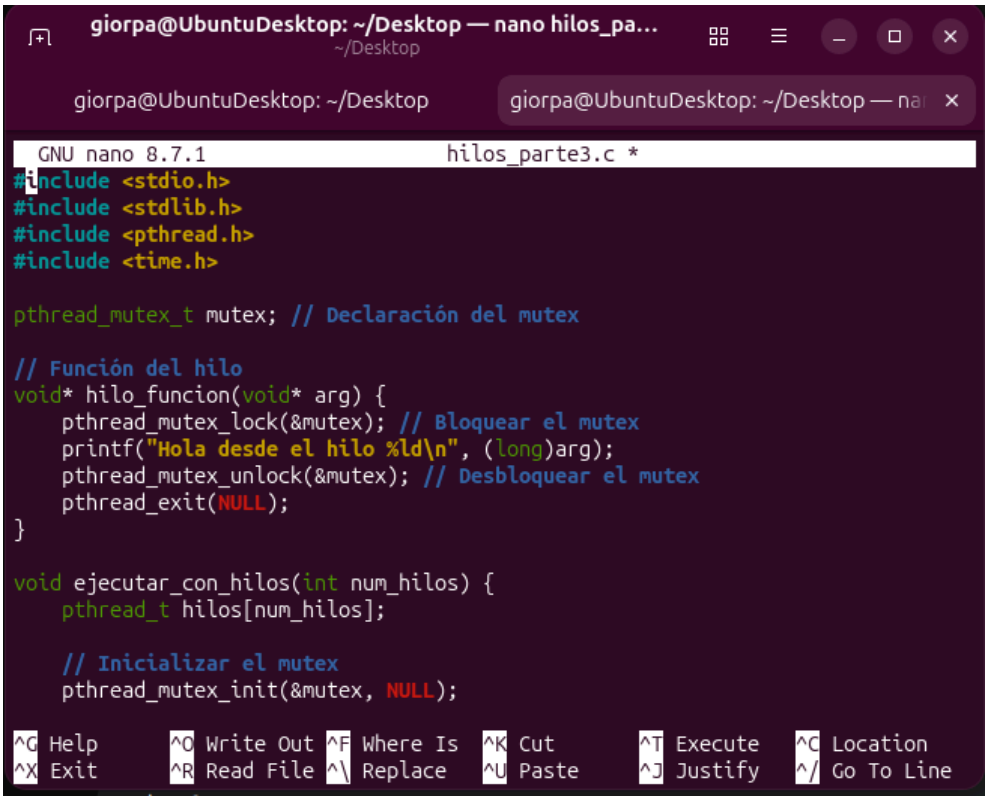
Parte 2: Implementación de sincronización

```
giorpa@UbuntuDesktop:~/Desktop$ gcc -o hilos_parte2 hilos_parte2.c -lpthread
giorpa@UbuntuDesktop:~/Desktop$ ./hilos_parte2
time ./hilos_parte2
Hola desde el hilo 2
Hola desde el hilo 1
Finalizó la ejecución del programa principal
Hola desde el hilo 1
Hola desde el hilo 2
Finalizó la ejecución del programa principal

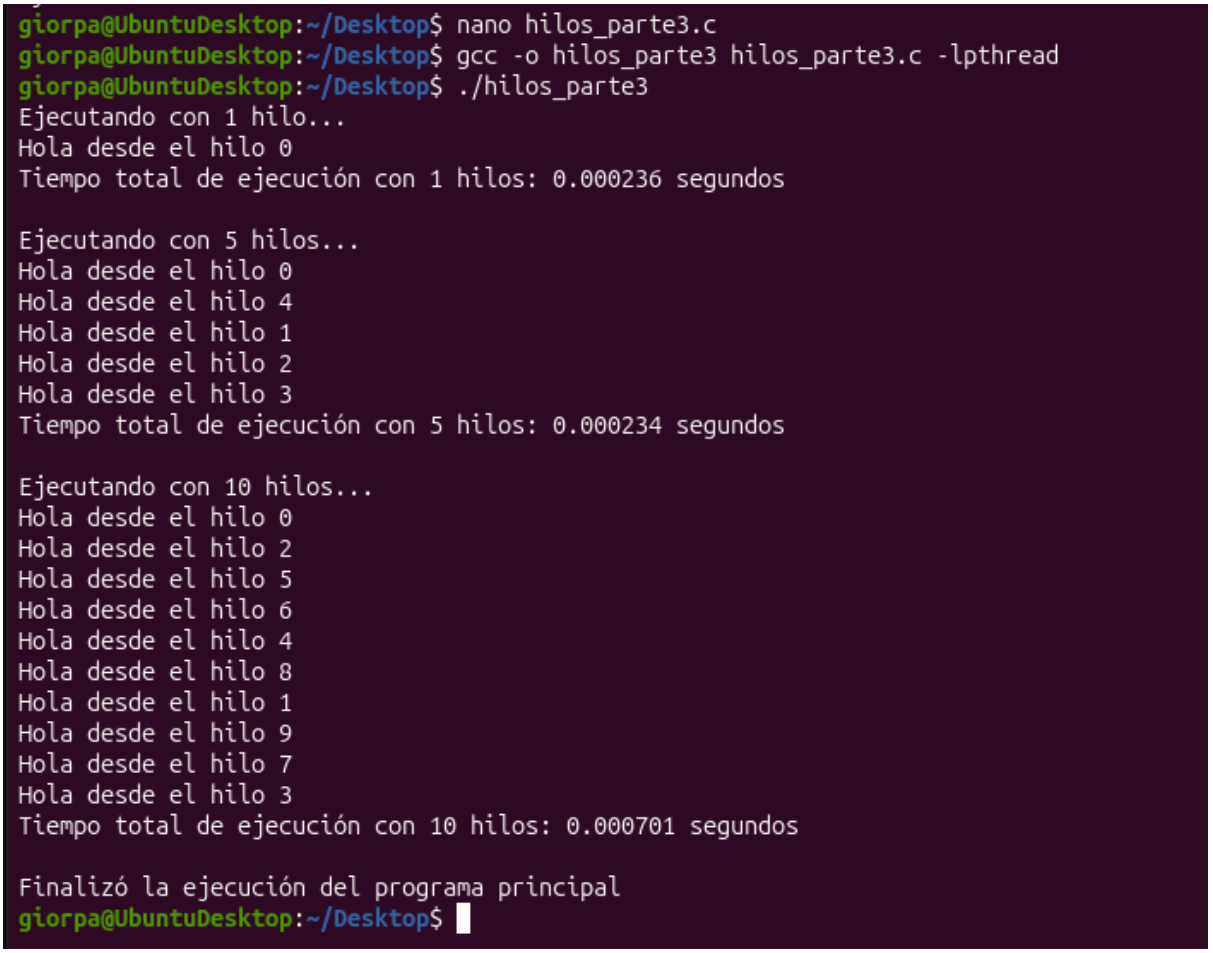
real    0m0.002s
user    0m0.000s
sys     0m0.001s
giorpa@UbuntuDesktop:~/Desktop$
```

Nota. Creamos un programa más y hacemos la actualización para la sincronización y vemos el registro del tiempo de ejecución

Parte 3: Análisis de rendimiento



Nota. Abrimos la terminal y con ayuda del editor de texto nano guardamos la actualización



Nota. Compilación y ejecución del programa con el registro del tiempo de ejecución

Parte 4: Análisis de resultados

1.- Al comparar el rendimiento de un programa multihilo con un monohilo, ¿qué diferencias observaste en el tiempo de ejecución? ¿Cómo influye la gestión adecuada de hilos en el rendimiento general del programa?

Aunque no sea una diferencia tan grande si se puede apreciar que en tareas muy pequeñas, el programa monohilo suele mostrar un tiempo de respuesta ligeramente menor, en arquitecturas multicore, la gestión adecuada de hilos permite repartir la carga computacional entre múltiples núcleos de procesamiento de manera simultánea, incrementando sustancialmente el rendimiento

2.- ¿Qué funciones de la biblioteca pthread utilizaste en tu implementación? Explica brevemente la finalidad de al menos tres de estas funciones y su importancia en la gestión de hilos.

- pthread_create: Crea e inicia un nuevo hilo en el sistema asociándolo a una rutina o función específica.
- pthread_join: Evita que el programa principal finalice antes de que sus hilos de trabajo terminen.
- pthread_mutex_lock / pthread_mutex_unlock: Garantizar que solo un hilo acceda a la sección crítica a la vez, garantizando la consistencia de los datos.

3.- ¿Cómo se comportó el programa al aumentar el número de hilos más allá de cierto punto? ¿Hubo un límite en el que el rendimiento no mejoró o incluso disminuyó? Explica por qué esto puede ocurrir en un entorno multihilo.

Al aumentar el número de hilos de 1 a 5 y luego a 10, se observa que el tiempo transcurrido de CPU o sobrecoste no disminuye de forma indefinida, existe un límite práctico determinado por el número de núcleos físicos o lógicos de la CPU del sistema/máquina virtual, cuando el número de hilos supera la cantidad de núcleos disponibles, el sistema operativo se ve obligado a realizar constantes cambios de contexto y competir por la serialización del cerrojo *mutex*, lo cual incrementa la latencia global y degrada el rendimiento.

4.- Durante la ejecución, ¿observaste alguna condición de carrera o comportamiento inesperado? Si es así, ¿cómo impactó esto en los resultados obtenidos y qué medidas se podrían tomar para prevenirlo?

Sin mecanismos de sincronización, varios hilos intentando acceder o modificar recursos compartidos, la inclusión de un *mute*x (pthread_mutex_t) sirvió para serializar el acceso a la instrucción de salida (printf), asegurando una ejecución ordenada en la sección crítica, para prevenir condiciones de carrera en general, se debe recurrir a primitivas de sincronización apropiadas como mutexes, semáforos o variables de condición